SYSAD Centralized Management & Access Control Portal

System Architecture, Core Functions, DFD Alignment & Technical Change Ledger

Architecture: 4 DFD Processes • 2 Relational Databases • 4 RBAC Actors • 6 Expected Outcomes • Telemetry Agent

1. Architectural Foundation & DFD Process Alignment

The system is engineered directly from the Systems Administration Data Flow Diagram (DFD), implementing 4 core processes and 2 persistent SQLite databases:

DFD Component	Type / Scope	Functional Implementation & Capabilities
1.0 RBAC Policy Engine	Process (Auth)	• Granular permissions for Student, Faculty, IT Staff, and Administrator. • Session management with salted SHA-256 password hashing and token expiry. • Real-time dynamic permission matrix viewer on Admin dashboard.
2.0 Identity Directory	Process (Lifecycle)	• User lifecycle transitions: Active, Aging, and Suspended. • Automated batch cohort onboarding for multi-student intake. • Automated orphan deactivation for inactive profiles (>120 days).
3.0 Trouble Ledger	Process (Ticketing)	• User problem submission across Network, LMS, Hardware, Account, Facility. • Priority triage queue (Critical, High, Medium, Low) for IT Support. • Role-based ticket isolation (students/faculty see only own tickets).
4.0 Security & Performance	Process (Monitoring)	• Live server metrics: CPU load, RAM consumption, and Disk volume usage. • Mandatory credential rotation enforcement (>90 days aging). • Real-time perimeter firewall block alerting and incident ticket logging.
D1: User & Roles DB	Data Store (`user_roles.db`)	Stores user profiles, credential hashes, RBAC permission matrices, departments, login timestamps, and active sessions.
D2: Audit & Ticket Logs DB	Data Store (`audit_ticket_logs.db`)	Stores immutable security audit events, trouble tickets, maintenance tasks, server configs, firewall policies, and forbidden web filter rules.

2. Verification of the 6 Expected Outcomes

All six required systems administration operational outcomes are completely implemented and automated:

Expected Outcome	Operational Mechanisms & Technical Capabilities
Outcome 1: **Automated Account Management**	• **Batch Provisioning:** Instantly registers and activates student cohorts with unique IDs, auto-generated email addresses, and default departments. • **Orphan Cleanup:** Scans identity database and auto-transitions accounts inactive for >120 days into `Suspended` status.
Outcome 2: **Enhanced Network Security**	• **Granular RBAC:** Students and Faculty are strictly isolated from server governance, firewall rules, and foreign tickets. • **Orphan Token Revocation:** Active sessions are revoked immediately upon account status transition to `Suspended`.
Outcome 3: **Proactive Threat Prevention**	• **Mandatory Credential Rotation:** Automated background scan flags profiles where password has not been rotated in >90 days. • **Suspension Lockdown:** Denies portal access and rejects API tokens for all suspended accounts.
Outcome 4: **Accelerated Incident Resolution**	• **Prioritized Queue:** Outage tickets are prioritized (Critical > High > Medium > Low) for rapid IT triage. • **Permission Audit Trail:** Every RBAC update is immutably logged to accelerate root-cause analysis during security audits.
Outcome 5: **High Infrastructure Reliability**	• **Automated Snapshot Archive:** One-click backup archives physical `.bak` snapshots of both databases to `data/backups/`. • **Rapid Recovery Pipeline:** Verified one-click restore reloads identity and audit state from backup in under 2 seconds.

Outcome 6: **Optimized IT Operations**	• **Perimeter Firewall Governance:** Administrators can dynamically toggle firewall rules (`ACTIVE` ↔ `DISABLED`). • **Dynamic Server Tuning:** Runtime parameters (thread pools, timeout thresholds, log levels) are adjusted without server downtime.

3. Client-Side Hardware Telemetry Agent

Web browsers have strict sandboxes preventing direct access to host hardware. To satisfy real-world hardware monitoring:

- **Node.js Telemetry Agent (`student_agent/agent.js`):** Standalone background daemon utilizing the `systeminformation` library.
- **Python Fallback Agent (`student_agent/agent.py`):** Standalone cross-platform daemon utilizing `psutil`.
- **Hardware Metrics Polling (Every 5s):** Host CPU Load % (`si.currentLoad()`), RAM Used vs Total (`si.mem()`), and Storage Volume Usage (`si.fsSize()`).
- **Fleet Monitor:** Securely transmits workstation metrics to `/api/monitoring/telemetry/agent-report` and displays live hardware cards on IT Staff and Admin consoles.
- **Privacy Hardening:** Hardware MAC addresses are completely removed from all dashboard tables for student workstation privacy.

4. Campus Web Filter & Prohibited Domain Governance

A full perimeter firewall web filter inspects external browser traffic and enforces campus acceptable-use policies:

- **Admin Policy Management:** Administrators can dynamically **Add**, **Suspend**, or **Delete** prohibited domain patterns in the `forbidden_websites` table.
- **Domain Normalization Engine:** Normalizes user input by stripping protocols (`http://`, `https://`), `www.`, ports, paths, and trailing dots (`roblox.com.` → `roblox.com`).
- **Perimeter Interception & 403 Block Page:** Disallows prohibited web traffic, dropping packets and returning the authentic **403 FORBIDDEN - Campus Perimeter Web Filter Block Page**.
- **Automated Incident Audit:** Generates unique incident tickets (`SEC-INC-XXXXXX`), records the student identity, IP, and timestamp into `audit_logs`, and pushes real-time toasts to IT Staff.
- **Student Violation Modal:** When a student triggers a violation, a high-priority security dialog pops up warning them of policy non-compliance.

5. External Browser Inspection & Testing Methods

The system supports multiple testing methods for external browsers (Microsoft Edge, Google Chrome, etc.):

- **Method 1: Direct Domain URL:** Navigate directly to `http://localhost:8000/<domain>` (e.g. `http://localhost:8000/youtube.com` or `/roblox.com`) to instantly view the authentic 403 block page.
- **Method 2: Student Portal Testing:** Use the built-in **Campus Web Security Gateway** card on the Student Dashboard to 'Test in Portal' or 'Launch in Edge / Chrome'.
- **Method 3: 1-Click Browser Proxy Batch (`test_edge_chrome_block.bat`):** Opens Edge or Chrome with `--proxy-server='http://127.0.0.1:8000'` showing the exact target URL without altering Windows system settings.

6. Production Deployment & Automation Artifacts

Turnkey infrastructure scripts provide rapid deployment across Linux, Windows Server, and Container environments:

- **`Dockerfile`:** Production-ready container image running Python under a secure non-root service account.
- **`docker-compose.yml`:** Single-command container deployment (`docker-compose up -d`) with persistent volume binding for `./data`.
- **`deploy_linux_server.sh`:** Automated Ubuntu/Debian deployment script configuring a `systemd` daemon (`sysad-portal.service`) and UFW firewall rule for port 8000.
- **`deploy_windows_server.bat`:** Windows Server configuration script opening inbound port 8000 in Windows Defender Firewall.
- **`unit_test.py`:** Comprehensive automated test suite with 9 verification suites ensuring 100% test coverage.

7. Technical Quality & Stability Improvements

- **Identity Sanitization:** Removed `(SysAdmin)` from database, profile badges, welcome banners, and JavaScript displays for clean display of **Elena Rostova**.
- **Resolved NameError Bug:** Added `import re` to `app.py`, completely resolving the socket crash that previously caused 'Failed to fetch' on login.
- **Dual-Stack Network Binding:** Server binds to `0.0.0.0`, ensuring reliable connectivity via `localhost`, `127.0.0.1`, and external LAN IP addresses.
- **Telemetry Polling Optimization:** Disabled background telemetry polling on the login view to prevent unauthenticated error toasts.
- **Self-Proxy Recognition:** The gateway server now cleanly differentiates requests to the local portal from external web filter targets.